Journal of Manufacturing Systems 59 (2021) 426–440

Contents lists available at ScienceDirect

Journal of Manufacturing Systems

journal homepage: www.elsevier.com/locate/jmansys

Technical Paper

A study on a Q-Learning algorithm application to a manufacturing assembly problem

Miguel Neves, Miguel Vieira, Pedro Neto *

Univ Coimbra, CEMMPRE, Department of Mechanical Engineering, Portugal

ARTICLE INFO

Keywords:
Reinforcement learning
Q-Learning
Assembly sequence
Optimisation

ABSTRACT

The development of machine learning algorithms has been gathering relevance to address the increasing modelling complexity of manufacturing decision-making problems. Reinforcement learning is a methodology with great potential due to the reduced need for previous training data, i.e. the system learns along time with actual operation. This study focuses on the implementation of a reinforcement learning algorithm in an assembly problem of a given object, aiming to identify the effectiveness of the proposed approach in the optimisation of the assembly process time. A model-free Q-Learning algorithm is applied, considering the learning of a matrix of Q-values (Q-table) from the successive interactions with the environment to suggest an assembly sequence solution. This implementation explores three scenarios with increasing complexity so that the impact of the Q-Learning's parameters and rewards is assessed to improve the reinforcement learning agent performance. The optimisation approach achieved very promising results by learning the optimal assembly sequence 98.3% of the times.

1. Introduction

Alongside the advent of product customisation, industrial manufacturing processes are increasingly more complex, required to be highly flexible, resourceful, and efficient. Their related decision-support problems have been explored in literature with the development of traditional optimisation mathematical models (e.g. LP, IP, MILP or MINLP) and heuristic approaches [1], and where machine learning algorithms have been gathering particular relevance. However, most machine learning approaches rely on trial and error interactions that requires available data (e.g. supervised learning), which is often unavailable. A current approach to such optimisation problems is based on reinforcement learning (RL).

RL is a machine learning paradigm originally inspired by the way biological systems learn [2], where an agent (e.g. a human, a robot, a vehicle) interacts with the environment by taking actions, Fig. 1. As a consequence of the action taken, the environment defines states and rewards. A state is essentially a description of the situation of the environment and the rewards are an abstract concept that describes the feedback by which the success or failure of the agent is measured. This kind of learning is gathering relevance in problems that require decision making, by dynamically exploring a solution that maximises the total rewards.

The main advantages of RL compared to other machine learning approaches are the fewer data available in the learning phase, which are the current states and expected rewards in a continuous learning process, fuelled by the interaction between the agent and the environment. Given its characteristics, RL methods are used in complex problems where there appears to be no obvious or easily programmable solution, such as game playing, robotics, control problems or operational research, and in problems where there is not enough labelled data such as anomaly detection problems. Since most of programming tasks are tedious and require years of expertise, RL algorithms can be applied to replace it with an intuitive process comprehensible even by an unskilled user.

1.1. Reinforcement learning methods and applications

One of the most successful applications of RL in recent years was the development of the Go playing program known as AlphaGo. This program achieved a 99.8% winning rate against other Go programs and defeated the European Go champion by 5 games to 0 [3]. This program was trained by supervised learning from human expert moves and by reinforcement learning from self-play. This application was further improved (AlphaGo Lee) and was capable of defeating the 9 dan player Lee Sedol, winner of 18 international titles, 4 games out of 5 [4].

* Corresponding author.
E-mail address: pedro.neto@dem.uc.pt (P. Neto).

https://doi.org/10.1016/j.jmsy.2021.02.014
Received 8 January 2021; Received in revised form 18 February 2021; Accepted 23 February 2021
Available online 25 March 2021
0278-6125/© 2021 The Society of Manufacturing Engineers. Published by Elsevier Ltd. All rights reserved.

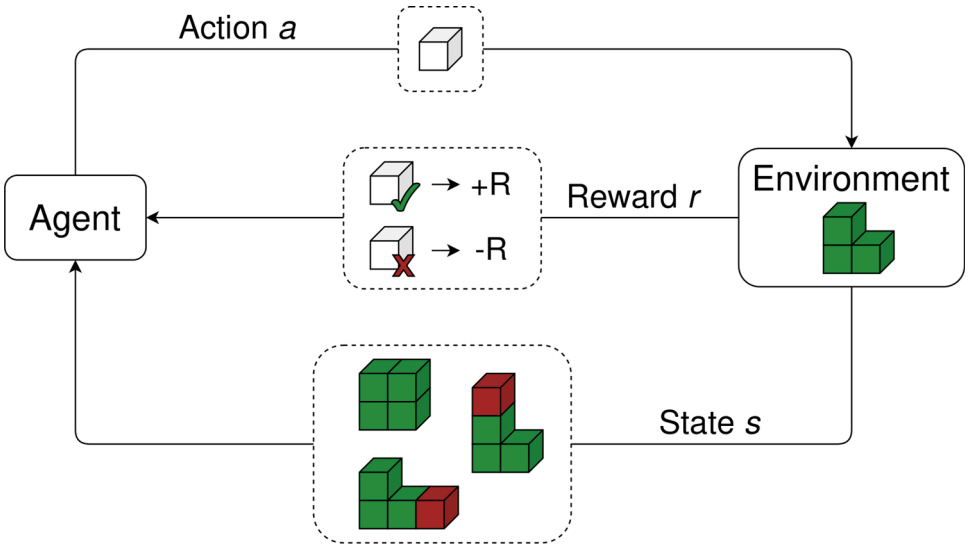


Fig. 1. Agent and environment interaction.

Also, in the field of game playing, RL was employed in a set of 49 Atari games by developing the DQN algorithm. This new algorithm was able to outperform the best RL methods at the time in 43 out of the set of 49 games. Furthermore, DQN performed at a level comparable to a professional player, achieving more than 75% of the human score in 29 out of the set of 49 games tested [5].

In recent years, the research on the applicability of RL is also increasing in the fields of decision-making and system control problems. By applying Q-Learning in a stock optimisation problem were achieved results up to 25% better when compared to traditional stock management algorithms, [6]. Wang and Usher studied the implementation of the Q-Learning algorithm for the usage of job agents when establishing routing decisions in a job shop environment, [7]. The authors discussed the effects of the Q-Learning application with the guidelines for future applications and recommendations for factor settings. Also, in dynamic job shop scheduling problem (DJSS) [8] proposed the usage of RL with a Q-factor algorithm to improve the scheduling method's performance while considering random job arrivals and machine breakdowns. In a simulated environment, this proposed method achieved high performances. In automotive paint shops, colour changeovers between consecutive production orders are a source of costs due to the process of cleaning painting robots. In order to minimise these costs, production orders can be re-sequenced and grouped in identical colours as a batch. Leng et al. propose the usage of the Deep Q-Network algorithm to solve this Colour-batching Re-sequencing Problem [9]. Huang et al. proved, through a simulation study, the effectiveness of the usage of Q-Learning in a maintenance problem where random failures of machines are highly disruptive [10]. The usage of reinforcement learning was also proposed to tackle the profit optimisation problem of a single product production affected by deterioration failures, requiring both maintenance and repairs [11]. The performance in manufacturing work cells that utilise gantries to load and unload the materials and parts needed is highly dependent on the gantry movements in real operation. Ou et al. formulated the gantry scheduling problem as a RL problem through the usage of Q-Learning and demonstrated, from the simulation results, the capability of effectively reducing system production losses in real-time operations, [12]. In a later article, five different reward functions were devised based on different assumptions of the system. It was shown that the policy derived from systematic analyses of production loss significantly outperformed the other policies. Therefore, it was concluded that the level of understanding of the system and how this understanding is transmitted to the reward function greatly impacts the learning model's success [13]. In the areas of production manufacturing [14], RL has been proposed to improve assembly efficiency by a dual-arm robot and achieved higher performance when compared with other methods. In recent years, customised production has emerged due to the increasing customer demand for more personalised products. This type of production, when compared with traditional production, has more uncertainty and variability. Such problems can be tackled by the usage of multi agent systems and reinforcement learning in a smart manufacturing environment. This approach was shown to be competitive in a dynamic environment [15]. Deep learning and reinforcement learning have been used to order acceptance decision-making of customized electronics towards sustainable manufacturing (reduction of carbon consumption and improvement of material utilisation) [16]. The automation of shoemaking production lines is extremely challenging also due to the versatility of manual product customisation. To tackle this problem a cyber-physical system artificial intelligence architecture was devised for the complete manufacturing of soft fabric shoe tongues by using Deep-Q reinforcement learning as a means of achieving better control over the manufacturing process and convolutional and long short-term memory artificial neural network to enhance action speed [17].

In a collaborative assembly process context, [18] proposed an approach based on Interactive Reinforcement Learning to reduce the programming effort required by an expert. The learning approach is made in two steps. The first one consists of modelling simple tasks that constitute the assembly process, using task-based formalism. These modelled simple tasks are then used by the robotic system by proposing to the user a set of possible actions at each step of the assembly process via a graphic user interface (GUI). After the user selects the action, the robot performs it, progressing the assembly process while learning the assembly order. The framework also allows different users to teach different assembly processes to the robot. This proposed approach is based on Q-Learning and IRL and was successfully applied in a UR10 robot in an assembly process comprising tasks such as, picking, holding, mounting and receiving objects. Human-robot interactions have become abundant due to the increase of autonomous robotic operators. The consequence of such interaction is the introduction of a source of uncertainty and unpredictability from the human operator. Oliff et al. present a methodology for the implementation of a RL-based intelligent agent which allows a change in the robotic operator's behavioural policy in response to performance variation in the human operators [19]. Lastly, there has been an increase in the research of using digital twins in smart manufacturing environments. Xia et al. developed a control methodology named Digital Engine capable of acquiring process

knowledge, scheduling manufacturing tasks, identifying optimal actions and demonstrating control robustness [20]. Hu et al. proposed a new graphic convolution layer named Petri-net convolution layer (PNC). When utilizing DQN with a PNC network better solutions are obtained for dynamic scheduling problems [21]. Digital twins have been proposed to maximise the efficiency of different manufacturing systems, namely in large-scale automated high-rise warehouse product-service systems [22], in the design of flow-type smart manufacturing systems in Industry 4.0 [23] and in the parallel control of smart workshops operating under the mass individualisation paradigm [24].

1.2. Main challenges and objectives

Even though many advances were made in recent years, some major challenges where noted when applying RL algorithms [25] when applying RL algorithms. The first challenge arises in the learning phase where the agent must do a trade-off between doing actions known to be effective (exploitation) and actions not yet explored (exploration), which is known as the exploration-exploitation dilemma [2]. Another common challenge is the "curse of dimensionality" [26], where to ensure global optimality, data must be collected throughout the entire state-space, often infeasible in high-dimensional state-action spaces. The restrictions of expensive hardware, component's wear with economical and logistical consequences in the agent interactions with the physical world are known as the "curse of real-world samples". To reduce real-world interactions, a model can be used as a simulation system and the knowledge transferred to a real scenario. However, creating accurate models is very challenging and sometimes even impracticable. Small errors due to under-modelling can accumulate and make the simulation diverge from the real behaviour, which is known as "curse of under-modelling and model uncertainty". Finally, the desired behaviour in RL is often specified by the reward function, which is frequently easier than defining the behaviour itself, however, in practice, in some problems it may be astonishingly difficult. This RL challenge is often known as "curse of goal specification". The dynamic and complex nature of the manufacturing environment may lead to situations where the Q-table is too large, due to the increase of the state and action spaces. The usage of Q-learning in such situations might be unviable, and therefore deep RL algorithms such as DQN could be employed.

The proposed RL-based approach aims to study the exploration-exploitation dilemma and the curse of goal specification, in which the type of agent properties (e.g. learning rate or reward signal) can influence the results obtained, here tackled by the assessment of the algorithm's learning parameters in the solution's outcome. In addition, this intends to support work planning with the demonstration of the RL method application towards a feasible and efficient learning of an assembly sequence of a product. Therefore, it results in an innovative method to the optimisation of an assembly problem for a given object considering a time efficient solution. The solution performance is compared with other methods for a given case study.

The remainder of the paper is structured as follows: in Section 2, the problem formulation and modelling theory of the Q-learning algorithm is outlined; in Section 3, the case study is presented with the discussion of the results implementation for a set of assembly scenarios; and finalising, in Section 4, with the conclusions and future work.

2. Model formulation

2.1. Problem description

Due to the market diversification of on-demand product attributes, current production systems are being required to deal with assembly flexibility, in which one or multiple resources is in charge of all steps of product customisation. The tasks sequencing represents one of the major components directly related to the efficiency of the assembly process. However, the assembly of a product is constrained by the product design and quality considerations, which often derives many possibilities in assembly sequence steps.

The problem considers a representation of an assembly job of a complex product containing different parts and tools, decomposed in a number of tasks which can be assembled in different sequences by an assigned resource. Along with the achievement of a feasible assembly process, improving the time efficiency is nontrivial due to the complexity of sequences' immediate/general time dependence of previous completed tasks. In order to cope with these issues, the goal of this work is to assess the effectiveness of an RL agent algorithm in the optimisation of task sequences based on real-time system states.

2.2. Baseline formulation approach

To provide a baseline comparison, the problem is firstly formulated under two of the most common benchmark modelling approaches, (i) a mathematical optimisation MILP model and (ii) a constructive heuristic algorithm.

2.2.1. Mathematical model

The mathematical model is formulated given the following notations of sets, parameters and variables:

The problem is defined as an assignment problem of a set of tasks $i \in I$ to a set of sequence steps $k \in K$ based on the concept of a general precedence model, setting in Eq. (1) the objective function as the makespan minimisation. The variable C_{ik} defines the completion time of task i at each step k, given the assignment of task i to sequence step k set by variable Y_{ik}. Considering that the last step $k = K^{last}$ comprises a feasible assembly combining all possible tasks, the total completion time of the optimal assembly sequence is minimised.

$$\text{Minimise} \quad C_{\max} \geq C_{ik} \quad \forall i, k = K^{\text{last}} \tag{1}$$

s.t.

$$\Sigma_i Y_{ik} = 1 \quad \forall k \tag{2}$$

$$\Sigma_k Y_{ik} = 1 \quad \forall i \tag{3}$$

$$Y_{ik} + Y_{i'k'} < 1 \quad \forall i, i' \in P_{ii'}, i \neq i', k < k', k \neq k' \tag{4}$$

$$C_{ik} \geq \tau_i Y_{ik} + (C_{i'k-1} + \Delta_{i'i} Y_{i'k-1})|_{k>1} + (\Sigma_{k'>k-1} \Sigma_{i'' \notin Q_{i''i}, i'' \neq i} \Delta_{i''i} Y_{i''k'})|_{k>2} \forall i, i' \notin Q_{i'i,k} \tag{5}$$

$$C_{ik} \geq 0 \quad Y_{ik} \in \{0, 1\} \tag{6}$$

Sets:	
i, i', i''	Tasks
k, k'	Sequence step
Subsets:	
$P_{ii'}$	General precedence tasks feasibility
$Q_{ii'}$	Immediate forbidden sequence of tasks
Parameters:	
τ_i	Average processing time of task i
$\Delta_{ii'}$	Processing time variation of task i' given the previous completed task i
Variables:	
Y_{ik}	Binary variable to assign task i to sequence step k
C_{ik}	Completion time of task i at each sequence step k
C_{max}	Makespan

Regarding the formulation, Eqs. (2) and (3) considers the allocation constraints to allocate one and only one task to one step, and Eq. (4) guarantees the general precedence rule of two tasks (under subset $P_{ii'}$). Eq. (5) defines the completion time for every task and according to production step, ensuring the timing between two consecutive stages, defined by the allowed assembly sequence given by $Q_{i'i}$. Besides the processing time of each task per stage τ_i, the model considers the

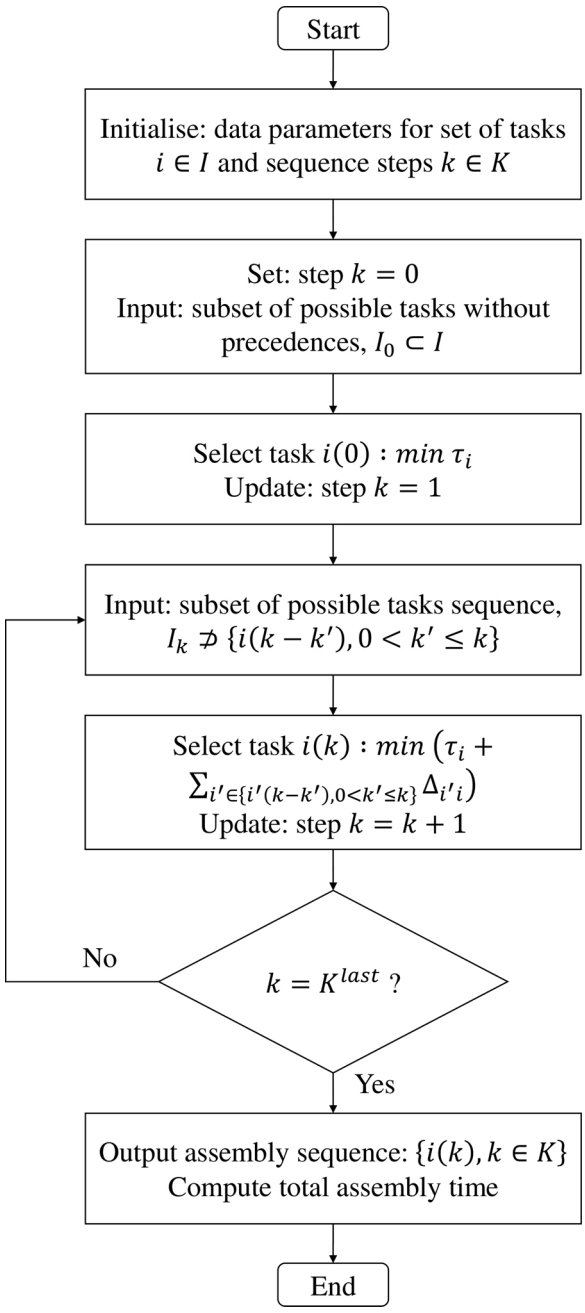


Fig. 2. Proposed heuristic algorithm structure.

variation of time duration (by penalizing (+) or rewarding (−) the related process time sequence) according to previous completed tasks, given by $\Delta_{ii'}$. Finally, Eq. (6) reassures the non-negativity and integrality of variables.

2.2.2. Constructive heuristic algorithm

Despite that the mathematical modelling approach offers an efficient solution search to guarantee the optimal decision for a given criterion, the computational hindrance commonly verified in large dimension problems requests a solution compromise. Instead, heuristic algorithms offer a method that cannot guarantee optimality, but a reasonable flexibility to reach an approximation in suitable computational time. For example, a constructive heuristic procedure starts with an empty solution and compiles an iterative set of predefined rules to reach a complete solution. For the given problem, a simple algorithm structure is devised in Fig. 2, in order to define a solution with the lowest assembly time for a given set of tasks $i \in I$ and sequence steps $k \in K$. The selection of the first task of the sequence is given by the lowest processing time τ_i of a subset of possible tasks with no precedences, I_k. As follows, an iterative procedure considers the lowest processing time combined with the corresponding time variation given the previous tasks sequence $\Delta_{ii'}$ under the subset of possible tasks, I_k. The algorithm ends with the completion of the predefined steps, $k = K^{last}$, defining the output of an assembly sequence and computing the total assembly time.

2.3. RL modelling theory

Given the anatomy of RL algorithms, they can be classified into three categories of methods which are value-function methods, policy search methods and actor-critic methods. The value-function methods, also known as critic-only methods, are based on the idea of initially discovering the optimal value function by fitting a value function or a Q-function and then deriving the optimal policy from this. On the other hand, the policy search methods, also known as actor-only methods, search directly in the policy space by summing the rewards of sample trajectories, which is only possible if the search space is restricted. In the particular case of policy search algorithms, known as policy gradient methods, one step of gradient ascent is applied to the expected reward objective. Lastly, the actor-critic methods are a combination of both where the critic's function is to monitor the agent's performance by fitting a value function or a Q-function to determine when the policy must be changed by the actor. Moreover, the models can also be divided into model-free algorithms, which do not use models of the environment and are explicitly trial-and-error learners, and model-based algorithms, which use the model for planning or policy improvement.

RL algorithms are commonly formalised as Markov decision processes, where the agent observes the current state and decides the next suitable action. The reason for this formalism is the increased difficulty of computation by considering all the states and actions taken from the initial state. Using MDPs, the system only needs to keep track of the last state and action. However, it is important to understand that this Markov assumption leads to the loss of data, which in some situations might

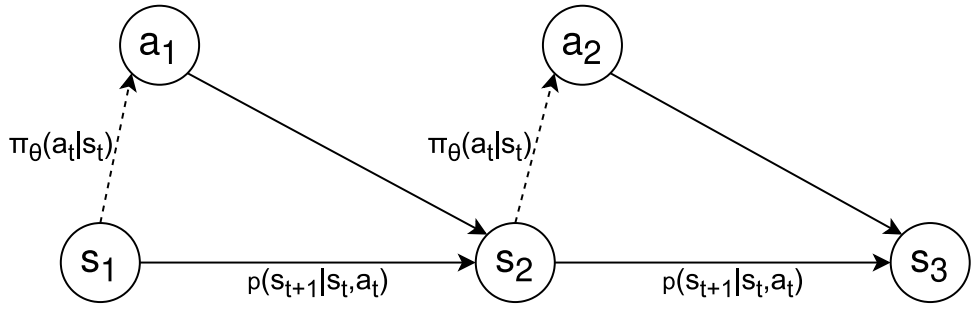


Fig. 3. Markov decision process (MDP).

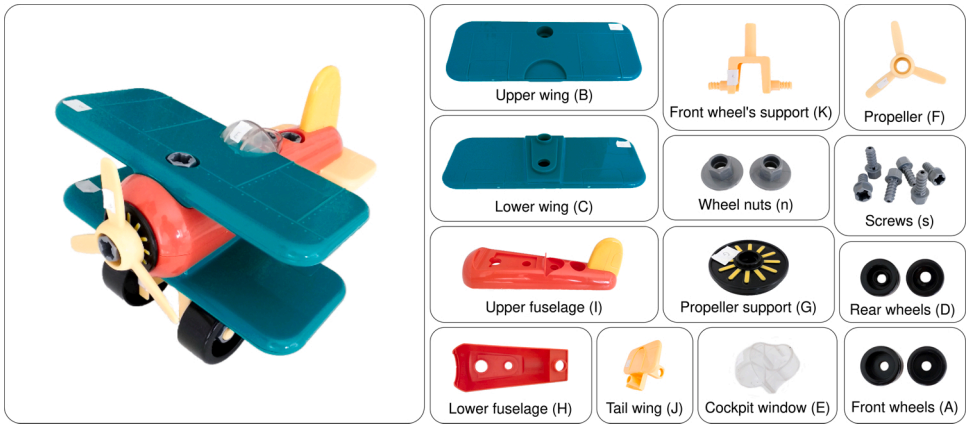


Fig. 4. Aeroplane from the Yale-CMU-Berkeley Object and Benchmark Dataset.

be relevant since rewards may be infrequent and delayed.

To define a Markov decision process, $M = \{S, A, T, r\}$, it is required to define a state space S, an action space A, a transition operator T and a reward function r. The state space is a set of valid states s the system can occupy, $s \in S$. The action space is the set of possible actions a the agent can take such that $a \in A$. In Markov decision processes the transition probabilities are not only conditioned on the previous state but also on the previous action, $p(s_{t+1} \mid s_t, a_t)$. Since the policy is the mapping of the states to actions, $\pi_\theta(a_t \mid s_t)$, a graphical representation of a Markov decision process can be observed in Fig. 3.

The learning agent must evaluate whether if it is being successful in the task. An agent can discern good from bad events based on the reward signal, which is analogous to the way the humans learn when experiencing pain or pleasure. This is the primary source of improvement of the policy since if the action selected in a certain state returns a low reward, then the policy may be changed so that, when faced by the same exact state, the policy selects a different and more rewarding action. However, the agent's goal is to maximise the accumulated reward over time through the actions chosen. However, an action with a high immediate reward might not be the optimal choice since it may lead to a lower accumulated reward over the future. To tackle this issue, there are two important concepts, the value-function and the quality function, usually known as Q-function. Given a policy, the value-function is defined as the total expected reward from a given state s_t:

$$V^{\pi}(s_t) = \sum_{t'=t}^{T} E_{\pi_\theta}[r(s_{t'}, a_{t'}) | s_t] \tag{7}$$

The Q-function is, on the other hand, the total expected reward from taking the pair action a_t in the state s_t:

$$Q^{\pi}(s_t, a_t) = \sum_{t'=t}^{T} E_{\pi_\theta}[r(s_{t'}, a_{t'}) | s_t, a_t] \tag{8}$$

Actions must therefore be selected based on value judgements because the agent's goal is to maximise the accumulated reward over time. Unfortunately, while rewards are given directly by the environment, values must be estimated multiple times within the sequence of observations.

2.3.1. Q-Learning

In order to implement the algorithm, it is necessary to understand the Q-Learning parameters used, such as how the Q-table is updated. The value iteration update is done at each step through the Bellman Equation, which consists on the weighted average of the old Q-value and the new information obtained, where α corresponds to the learning rate, γ to the discount factor, r_t to the received reward when moving from state s_t to s_{t+1}, $Q^{new}(s_t, a_t)$ to the new Q-value of the state s_t and action a_t, $Q(s_t, a_t)$ to the old Q-value of the state s_t and action a_t and $\max_a Q(s_{t+1}, a)$ to the estimate of the optimal future Q-value:

$$Q^{\text{new}}(s_t, a_t) \longleftarrow Q(s_t, a_t) + \alpha \times [r_t + \gamma \times \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \tag{9}$$

The learning rate parameter has values between 0 and 1 and influences the extent on how the new information changes the current state value, which means that a lower learning rate leads to a longer learning time. However, it is important to note that a higher learning rate may lead to suboptimal results or even divergence in non-deterministic scenarios. The discount factor determines the importance of future rewards, so the lower its value the less meaningful are the future rewards. If the discount factor has the value 0, only the current reward is considered. The selection of the action is made using an epsilon greedy search, i.e. the agent selects a random action with probability ε and otherwise selects the action greedily, with probability $1 - \varepsilon$, by selecting the action with the highest Q-value. The value of epsilon (ε) decays based on a decay rate known as epsilon decay. The Q-Learning algorithm is summarised in Algorithm Q-Learning.

Algorithm Q-Learning

```
Q(s, a) initialised arbitrarily
for each episode i do
    state s initialised
    for each step j do
        action a chosen from state s using policy derived by Q (e.g. ε-greedy)
        action a taken
        reward r and state s' observed
        Q(s, a) = Q(s, a) + α × [r + γ × max_a' Q(s', a') − Q(s, a)]
        s = s'
    end for
end for
```

2.3.2. Proximal Policy Optimisation (PPO)

The PPO algorithm was also implemented in order to evaluate the performance of the Q-Learning algorithm in the assembly decision making problem. PPO is a new family of policy gradient methods for RL (actor-critic), that alternates between sampling data by interacting with the environment and optimising a 'surrogate" objective function by using stochastic gradient descent (SGD). This new family of methods follows the ideas of the Trust Region Policy Optimization (TRPO) algorithm to attain a reliable performance by constraining the policy update size, and has the advantage of being relatively easier to use when compared with other deep RL algorithms. However, theoretically, it should be less sample efficient than Q-Learning. As summarised in the Algorithm PPO [27], the policy $\pi_{\theta_{old}}$, where θ_{old} is the vector of parameters before the update, is ran for T timesteps for each iteration and N

Table 1
Aeroplane's parts.

Part	Aeroplane part's description	Number of parts
A	Front wheels	2
B	Upper wing	1
C	Lower wing	1
D	Rear wheels	2
E	Cockpit window	1
F	Propeller	1
G	Propeller's support (engine)	1
H	Lower body of the aeroplane (lower fuselage)	1
I	Upper body of the aeroplane (upper fuselage)	1
J	Rear body of the aeroplane (tail wing)	1
K	Front wheel's support	1

Table 2
Aeroplane's fasteners.

Fastener	Fastener's head type	Number of fasteners
n	Wheel nuts	2
s	Screws	5

parallel actors set. After this data sampling phase, the advantage estimates $\widehat{A_T}$ are computed. This estimates are given by the difference between the discounted sum of rewards obtained by the agent and the value function, i.e. the estimated return when starting in the state *s*. Then the surrogate loss function *L* is optimized with respect to the policy parameters in minibatches of samples for *K* number of epochs. Finally, the policy parameters are updated [27].

Algorithm PPO, Actor-Critic Style

```
for iteration=1,2,... do
  for actor=1,2,...,N do
    Run policy π_θold in environment for T timesteps
    Compute advantage estimates Â_1, ..., Â_T
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size M ≤ NT
  θ_old ← θ
end for
```

3. Experiments and results

3.1. Case study

The object chosen for the assembly problem is an aeroplane toy from the Yale-CMU-Berkeley Object and Benchmark Dataset [28,29] (Fig. 4), which process is studied and optimised through the implementation of a RL methodology.

As follow, the object components and assembly structure are thoroughly analysed. As a preliminary evaluation, the problem solution is derived from the mathematical optimisation model. Then, the assembly problem is formulated using Q-Learning and is implemented in different scenarios where the agent must learn a feasible assembly sequence, i.e. an assembly sequence that respects all the task precedences. The scenarios consider distinct features incorporated in the decision-making algorithm to foment the learning of an optimised time efficient assembly sequence. The algorithm's learning parameters are individually analysed in order to improve the agent's performance with other methods.

3.2. Assembly analysis

The aeroplane object is comprised of 9 structural parts and 2 types of fasteners, which are displayed in Tables 1 and 2.

After subdividing the aeroplane into parts and fasteners, the assembly process was subdivided in a total of 8 tasks. In Table 3 each task is associated with the corresponding parts and fastener required. It is important to note that some parts can be used in more than one task. For the assembly to be complete, every task must be executed without repetitions, therefore the number of different assembly sequences is $n!=8!=40,320$.

However, the feasible number of assembly sequences is lower than the previously calculated one, due to the fact that certain tasks require other tasks to be previously completed. Such precedence sequence dependencies are displayed in Table 4. When taking into account the task dependencies, the feasible number of assembly sequences is 3360, which corresponds to only 8.3% of all assembly sequences.

In resume, the assembly process is subdivided in 8 different tasks, or actions, and the assembly can be considered as complete when all the 8 tasks have been executed. Therefore, the MDP's states can be defined by the updated assembly status at each task, where the initial state would correspond to the situation where none of the actions was executed, the final state when all the actions were completed and the aeroplane is assembled, represented by the binary number indicating whether the task has been executed. The digit in the binary number corresponds to the task number. If the task *n* has been executed, 1 is set to the *nth* digit in the binary number. Otherwise, the *nth* digit is set as 0 (Fig. 5). The initial state would then be represented as 00000000 and the final state would correspond to 11111111, which in decimal notation corresponds to 0 and 255 respectively, thus, the number of states is 256. Similarly, due

Table 4
Precedence task's feasibility, $P_{ii'}$.

Task	Precedence task
1	None
2	1
3	1
4	1
5	1 and 4
6	1
7	None
8	None

Table 3
Parts and fasteners associated to each task and their respective quantities.

Tasks	Parts											Fasteners	
	A	B	C	D	E	F	G	H	I	J	K	n	s
1							1	1	1				
2										1		1	
3						1						1	
4			1									1	
5		1									1	1	
6		1			1							1	
7	1										1		2
8				1						1		1	

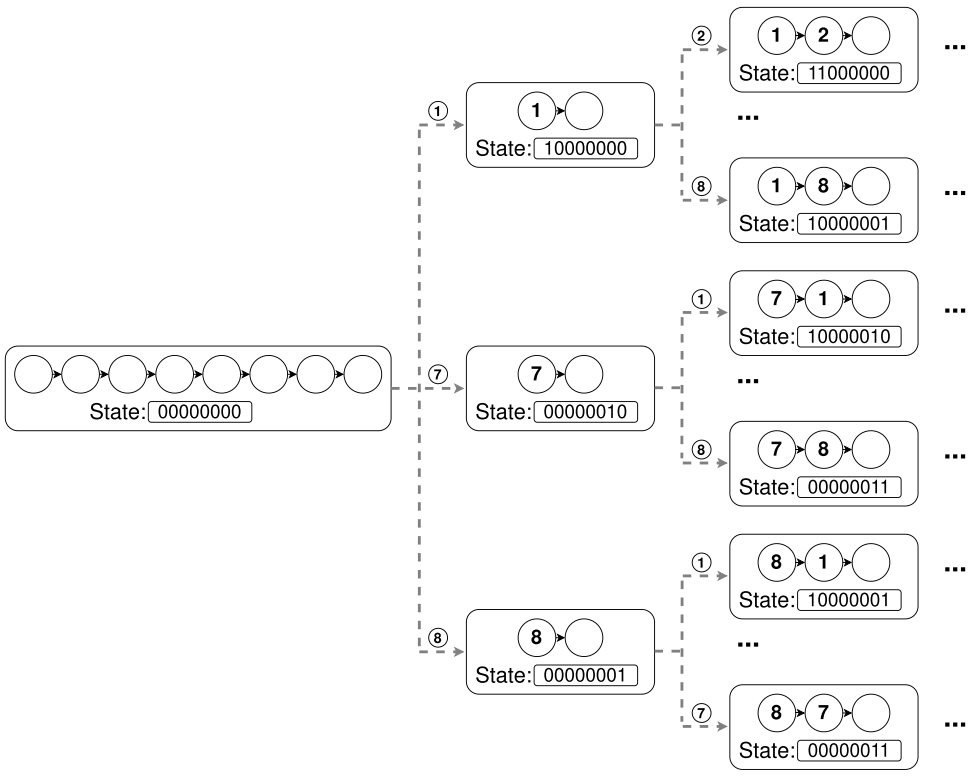


Fig. 5. MDP's states and actions scheme.

Table 5
Task's average time, τ_i.

Task	1	2	3	4	5	6	7	8
Average time [time units, t.u.]	10	7	8	6	12	8	11	9

Table 6
Tasks' variation in respect to the average time given completed tasks $\Delta_{ii'}$ [t.u.] and immediate forbidden sequences, $Q_{ii'}$.

Task	1	2	3	4	5	6	7	8
Task 1 done							0	0
Task 2 done			−1	−1.5	0	−1	0	1
Task 3 done		0		0	0	0	0	0
Task 4 done		−0.5	0			0	0	0
Task 5 done		−1	−0.5			−2	1	0
Task 6 done		0	0	0	0		0	0
Task 7 done	0	0	0	0	0	0		0
Task 8 done	0	0	0	0	0	0	0	

to task precedencies, there are only 100 possible states.

Finally, for this case, the tasks' average time were estimated τ_i (Table 5) as well as the increase/decrease variances on the average times with respect to the tasks previously done $\Delta_{ii'}$ (Table 6). In this table is also shown empty cells with the immediate forbidden sequences, which add as restrictions to the previous general precedence.

3.3. Baseline solutions

To assess a baseline comparison of the Q-learning algorithm performance, two pilot solutions can be determined based in the proposed alternative modelling approaches. In one hand, considering the mathematical optimisation model defined in section 2.2, it is implemented in GAMS (29.1.1 ver.), using CPLEX (12.8.0.0 ver.) solver, run in an Intel (R) Core(TM) i7-7700HQ @2.80GHz with 16GB RAM. Given the problem description and data defined in Section 3.2, the optimal solution obtains an assembly task sequence $1 \rightarrow 8 \rightarrow 4 \rightarrow 7 \rightarrow 5 \rightarrow 2 \rightarrow 6 \rightarrow 3$ with the accumulated value of all tasks of 65 time units (t.u.). The solution complies with the given feasibility constraints for an optimality gap of 0 in under 1 second and represents the best assembly combination possible for the given case conditions, although multiple optimal solutions with the same assembly time duration can occur. On the other hand, regarding the constructive heuristic algorithm presented in Fig. 2, the iterative procedure obtained the assembly solution of tasks $8 \rightarrow 1 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 7 \rightarrow 5$ with an accumulated value of 68.5 t.u. As expected, the solution obtained with this algorithm also presents a feasible solution but with an assembly time far from the best optimal value solution given by the MILP model. These possible solutions offer two alternative assembly sequences for the given problem, in accordance with the chosen method as resumed in Table 7.

3.4. Scenario I: Learning an assembly sequence based on estimated task average times and variances with the Q-learning algorithm

To better understand the scenario's results of the Q-learning implementation using Matlab R2020a, it is important to introduce the concept of experiment. An experiment comprises the algorithm's learning phase and the result obtained at the end. The learning phase in an experiment takes place over various episodes dictated by the maximum number of episodes per experiment. In this specific case, an episode starts with no tasks done and ends when all tasks have been successfully completed or when the maximum number of steps has been reached. However, in all scenarios the algorithm considers that, whenever the Q-Learning agent selects an impossible action, the current state does not change, which means that the sequence is penalised for requiring more than 8 steps to complete an episode.

At the end of the experiment, the agent selects the actions based solely on the Q-Values, which means that after learning, the agent always selects the same assembly sequence (expressed as learned assembly sequence or experiment's result). In order to study statistically

Table 7
Solutions outputs.

MILP output	#Total variables 129	#Binary variables 64	#Equations 550	Optimality gap 0.0	CPU (s) 0.34	Solution value
Heuristic algorithm	Assembly sequence Assembly sequence	1 → 8 →4 → 7 →5 → 2 →6 → 3 8 → 1 →4 → 2 →3 → 6 →7 → 5				65.0 t.u. 68.5 t.u.

relevant solutions, each set of parameters and rewards is replicated in 120 experiments.

With the objective of understanding how well the agent would be able to compare the efficiency of the feasible assembly sequences, the rewards are assigned to the processing time of each task. These were then defined through Eq. (10), where $\mathbf{R}(s, a)$ is the reward for taking the action a in the state s, r_m is the reward multiplier, r_s the reward shift, r_p the reward penalty and $\mathbf{T}(s, a)$ the predefined time it takes to complete the action a in the state s, that is calculated by summing the respective variances to the task's average time:

$$\begin{cases} R(s,a) = r_m \times (-T(s,a) + r_s) & \text{if possible action} \\ R(s,a) = r_p & \text{if impossible action} \end{cases} \quad (10)$$

Since the RL algorithm's goal is to maximise the accumulated reward, in order to learn the most time efficient assembly sequence (minimise the assembly sequence time), the value matrix $\boldsymbol{T(s,a)}$ must be subtracted. The r_s shifts each reward by its value and, as a result, shifts the accumulated reward by eight times its value. The r_m, on the other hand, multiplies the shifted reward. Consequently, the accumulated reward is also multiplied by r_m. If r_m and r_s have the values of 1 and 0 respectively, the accumulated reward is equal, in absolute values, to the duration of the assembly sequence. The accumulated rewards for all feasible assembly sequences for these exact values of r_m and r_s are displayed in Fig. 6.

As it can be observed in Figure 5 (B), the most common accumulated reward, corresponding to 18.4% in all feasible assembly sequences, is −69.5, which correlates to the corresponding total assembly time. An example of such an assembly sequence is 8 → 1 →3 → 4 →7 → 2 →6 → 5 where the accumulated reward is $\Sigma R(s,\ a) = -9 - 10 - 8 - 6 - 11 - 6.5 - 7 - 12 = -69.5$. It is also possible to observe in Fig. 6 (A) that

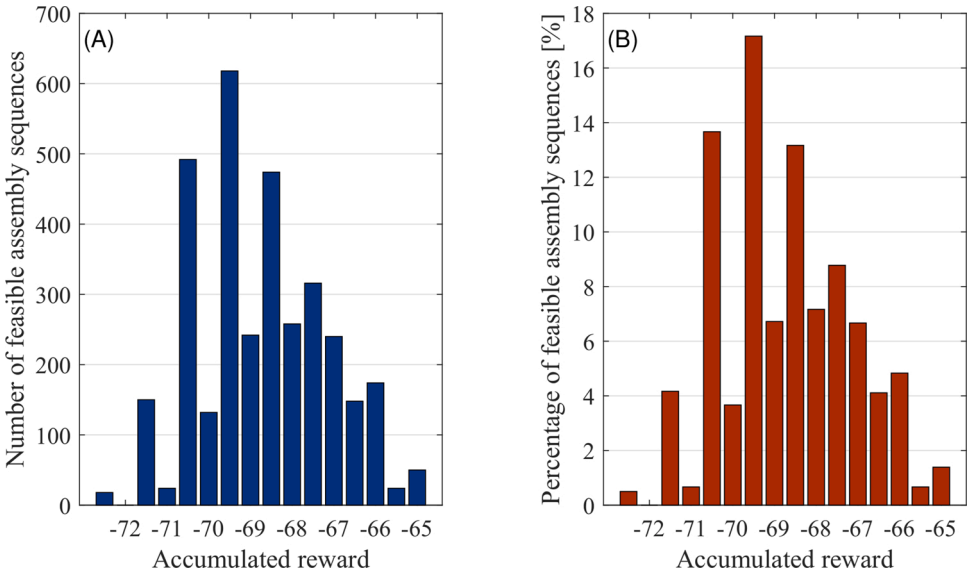


Fig. 6. Distribution by number (A) and percentage (B) of feasible assembly sequences' accumulated rewards.

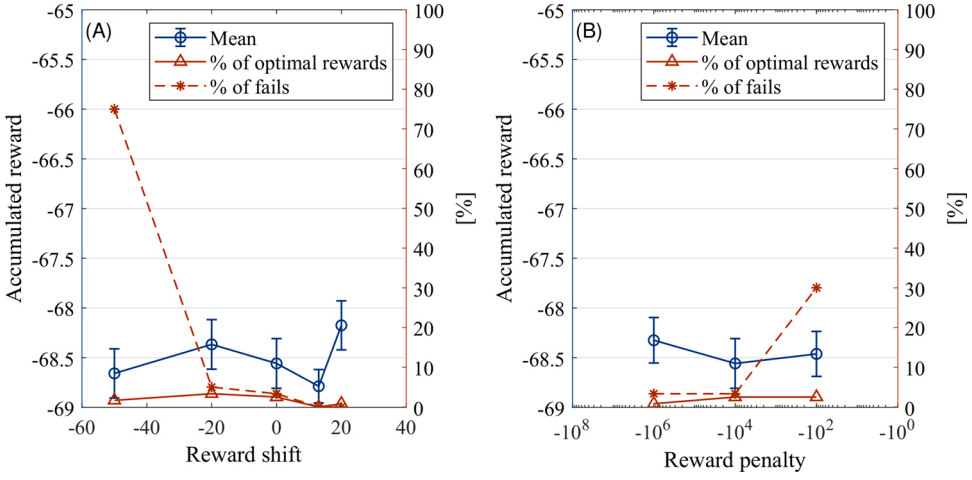


Fig. 7. Impact of the reward shift (A) and reward penalty (B) on the agent's performance.

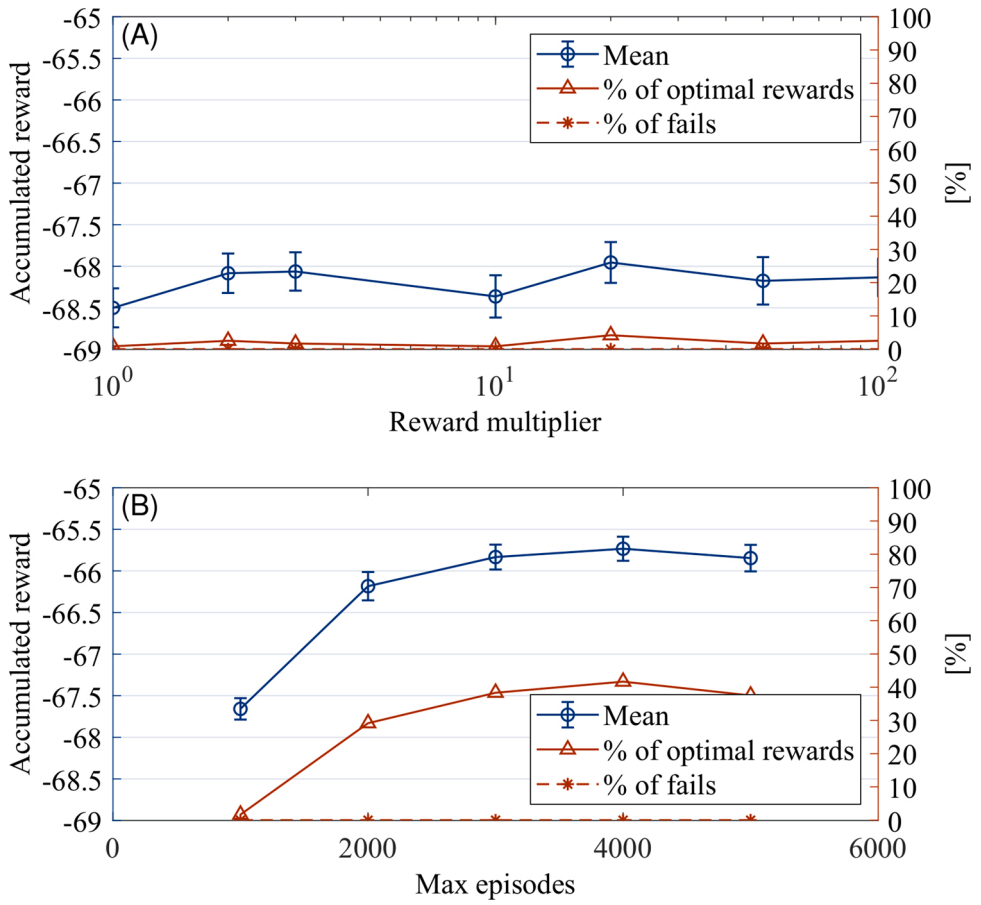


Fig. 8. Impact of the reward multiplier (A) and maximum number of episodes per experiment (B) on the agent's performance.

there are 50 feasible assembly sequences with the maximum accumulated reward of −65.0, which corresponds to the optimal accumulated assembly time shown by the mathematical optimisation model in Section 3.3.

In an initial sensitivity analysis, the rewards (r_s and r_p) were tested individually (Fig. 7) with the parameters in Table 6 and with the values 0, 1, and −10,000 for the r_s, r_m and r_p respectively. The comparison of the performances for the various sets of parameters and rewards considers three indicators: the mean accumulated reward, normalised for a r_m of 1 and a r_s of 0; the percentage of times the agent learned one of the 50 optimal assembly sequences; and the percentage of times the agent failed to learn one feasible assembly sequence in the 120 experiments. When the agent failed to learn a feasible assembly sequence, the number of experiments was increased so that the mean would reflect 120 correctly learned assembly sequences. The reason for this rule was the high penalty on the mean of an incorrectly learned assembly, which would turn unfeasible a correct comparison between sets.

When observing Fig. 7 (A), it can be concluded that a negative r_s value increases the likelihood of incorrectly learning an impossible assembly sequence because the agent is not able to differentiate between the penalties and the accumulated rewards shifted to values increasingly more negative. Also, a positive r_s may lead to better results, as seen in the mean increase for the value of 20. The reward penalty, as seen in Fig. 7 (B), understandably, influences the percentage of fails since an action with a higher r_p is more likely identified as an incorrect action, i.e. the bigger the penalty the lower the percentage of fails.

With the objective of understanding the impact of the r_m and the maximum number of episodes, they were individually changed maintaining the parameters and rewards of the previous sensitivity analysis, except for the r_s and r_p with the new values of 20 and −1,000,000 respectively. Also, in the experiments where the maximum number of episodes was altered, the selected r_m used was 5.

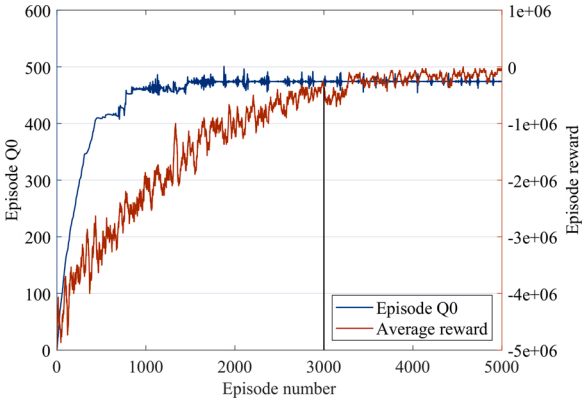


Fig. 9. Evolution of the episode reward over the episodes for a value of epsilon decay of 0.0001.

In terms of the r_m sensitivity analysis displayed in Fig. 8 (A), there are no statistically significant improvements in the agent's performance. However, the value chosen for r_m in future sets of parameters and rewards is 20 as it accomplished the best results. In respect to the maximum number of episodes, presented in Fig. 8 (B), it is possible to conclude that increasing the maximum number of episodes per experiment leads to an increase in performance (both visible in the mean and in the percentage of optimal accumulated rewards) until a certain value in which it seems to plateau. Though, is important to remember that a higher value for the maximum number of episodes is related to the amount of times the experiment has to be repeated for the agent to learn.

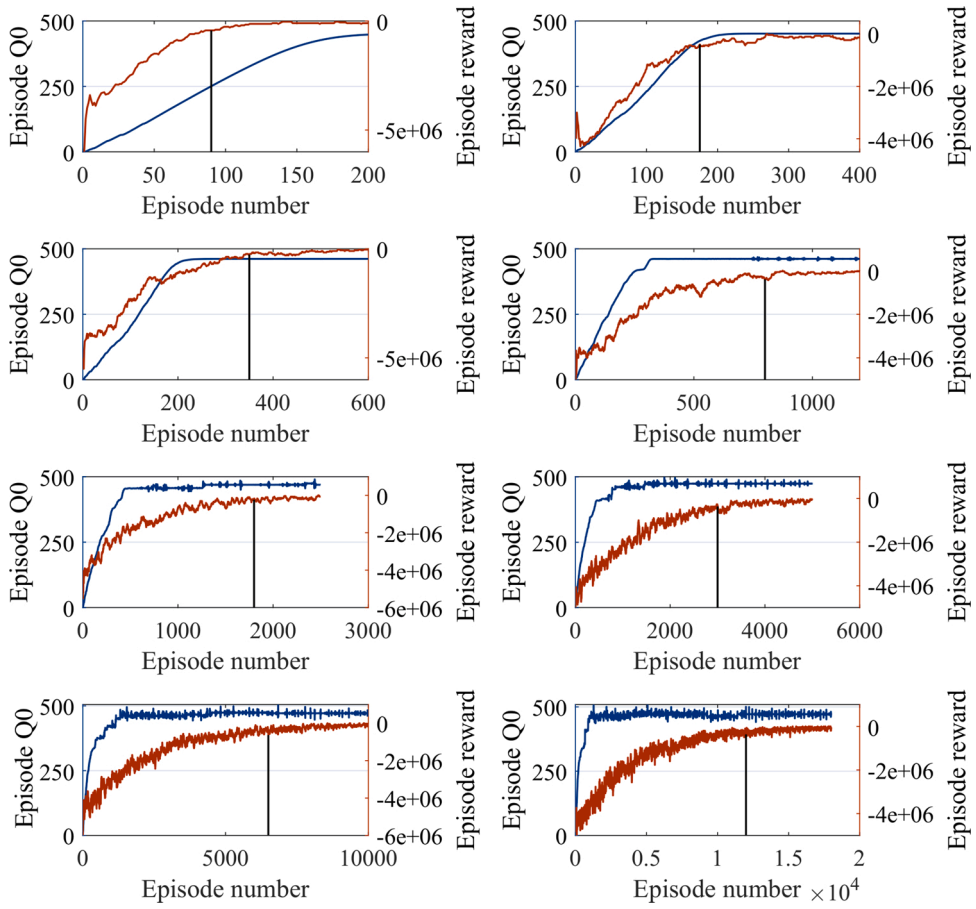


Fig. 10. Evolution of the episode Q0 (blue line) and episode reward (orange line) over the episodes for various values of epsilon decay. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 8
Maximum number of episodes selected for each epsilon decay value.

Epsilon decay	0.005	0.002	0.001	0.0005	0.0002	0.0001	0.00005	0.00003
Max episodes	90	175	350	800	1800	3000	6500	12,000

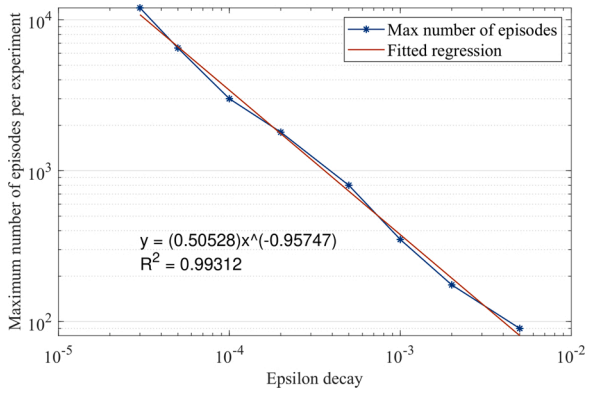


Fig. 11. Graph of maximum number of episodes over epsilon decay.

This means that it is essential to maintain the number as low as possible since in a real scenario it corresponds to the amount of assemblies required for the learning process. Thus, for these parameters, and specially for the epsilon decay of 0.0001, the optimal maximum number of episodes is 3000.

The optimal maximum number of episodes is dependent on the epsilon decay's value since a lower epsilon decay leads to a slower increase in the greedy selection by the agent and, as such, requires more episodes in the learning phase. In order to identify the relationship between these two parameters, a graph of the evolution of the episodic accumulated reward in one of the experiments (with 5000 maximum number of episodes) is analysed (Fig. 9).

When analysing Fig. 9, it is possible to observe that after the identified optimal maximum number of episodes the episodic accumulated reward does not greatly increase. This could explain why increasing the maximum number of episodes further does not lead to a significant

Table 9
Parameters for the pair epsilon decay and maximum number of episodes experiment.

Parameter	Value
Learning rate	1
Discount factor	1
Epsilon	0.9
Max steps per episode	8
Reward shift	20
Reward multiplier	20
Reward penalty	−1,000,000

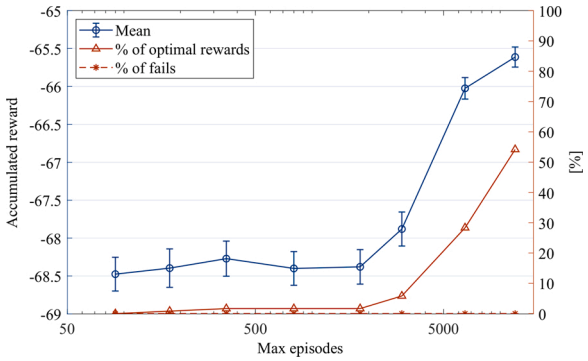


Fig. 12. Impact of the pairs of epsilon decay and maximum number of episodes on the agent's performance.

change in the results in Fig. 8. Similar graphs were analysed for different values of epsilon decay in order to identify in which episode the episodic accumulated plateaus during an experiment to select this value as the optimal maximum number of episodes per experiment for the given epsilon decay (Fig. 10 and Table 8).

From the values available in Table 8, a linear regression was devised using a logarithmic scale in both axes. As shown in Fig. 11, the data approximated fits the function $y = 0.50528 \times x^{-0.95747}$ where y is the maximum number of episodes and x the epsilon decay.

Additional new experiments were run with the parameters shown in Table 9, apart from the epsilon decay and maximum number of episodes which were based on Table 8. Results are displayed in Fig. 12.

As can be observed in Fig. 12, the increase in the maximum number of episodes, accompanied by the respective decrease in the epsilon decay, leads to better results with an increase in the mean and in the percentage of optimal rewards. It is also possible to notice that such increase only starts around 1800 and 3000 maximum number of episodes.

3.5. Scenario II: Learning an assembly sequence based on measured task average times and estimated variances with the Q-learning algorithm

In the second scenario, task time input data was measured and repeated 10 times (Table 10) so that the average processing times are now confirmed (approximation is used for simplification).

As in scenario I, the tasks' average times have variations in respect to the corresponding precedence tasks (Table 11). It is important to notice that the tasks variability complexity has increased from the scenario I, so that there would be a larger variety of accumulated rewards and a lower number of assembly sequences with the largest accumulated reward, i.e. optimal assembly sequences.

The variability was also increased by introducing the tool changeover time. Since there are two different types of fasteners, the fastening device's tool must be switched during the assembly process. Regarding the fastening device, there are two main assumptions made: the assembly process starts without any tool placed and the tool changeover lasts three time units to be performed. In order to introduce the tool changeover, the number of states must be increased. The states, apart from the binary number that define the completion of each task, can have three possible indexes (0, 1 and 2). The index 0 indicates that the fastening device does not have any tool placed (start of the assembly), the index 1 indicates that the fastening device has the screwdriver applied, and the index 2 indicates the nut driver is applied. The new total number of states is 511, since in the first state the fastening device has no tool and in any other state it can have either the first or the second tool. Due to task dependencies and tool selections, the number of possible states is 149.

With the respective changes to the average times and time variances the new distribution of accumulated rewards from the feasible assembly sequences can be observed in Fig. 13.

The new distribution has, as previously stated, a larger variance of accumulated rewards and the highest accumulated reward or optimal accumulated reward is shared only by 2 assembly sequences (Fig. 13 (B)), which are $7 \rightarrow 1 \rightarrow 8 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 3$ and $7 \rightarrow 8 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 3$, and has the value of -64. In this new scenario, it may be easier to understand the impact of the set's parameters on the agent's performance. The reward shift was modified for the values (0, 3, 6, 7, 8, 9, 10 12, 15) while using the values of Table 9, apart from the epsilon decay and maximum number of episodes, which had the values of 0.00005 and 6500 respectively. The results of the multiple sets of 120 experiments are displayed in Fig. 14.

The mean of all the 37 possible values of accumulated reward is -73 (for a r_s of 0 and r_m of 1), and therefore, if subdivided evenly, each task would have a reward of -9.125, which we will define as mean task reward. A value of r_s equal to the mean task reward shifts the accumulated rewards to a position where they are evenly separated into positive and negative. When analysing Fig. 14 it is possible to identify that the best accumulated reward occurs for a value of the reward shift of 8. Also, it is important to notice that the percentage of fails decreases with the increase of the reward shift and is approximately 0 for values higher or equal to 9. Thus, the optimal reward shift may be related to the mean

Table 11
Tasks' variation in respect to the average time given completed tasks $\Delta_{ii'}$ [t.u.] and immediate forbidden sequences, $Q_{ii'}$.

Task	1	2	3	4	5	6	7	8
Task 1 done							0	0
Task 2 done			−2	−3	−0.5	−2	0	1.5
Task 3 done		0		0	0	0	0	0
Task 4 done		−1	0			−1.5	0	0
Task 5 done		−2	−1			−3	2	0
Task 6 done		−1	−0.5	1	−0.5		0	0
Task 7 done	0	0	0	0	0	0		0
Task 8 done	0	0	0	0	0	0	0	

Table 10
Task's time measurements.

Task		1	2	3	4	5	6	7	8
Measured tasks [time units, t.u.]	1	6.36	9.10	8.59	6.75	10.62	9.87	12.36	10.06
	2	6.28	9.15	10.30	8.22	10.52	11.31	12.39	8.50
	3	4.71	8.00	8.70	7.90	9.82	11.56	12.88	8.03
	4	5.38	8.15	10.49	8.29	9.25	11.00	10.49	9.09
	5	5.80	8.30	8.75	7.25	9.44	8.95	11.22	8.90
	6	6.71	8.52	8.97	7.40	10.04	12.16	11.04	9.37
	7	5.73	8.17	9.00	7.95	9.07	9.24	11.94	7.70
	8	7.31	8.05	8.29	6.30	10.76	10.01	10.34	8.70
	9	5.16	7.62	8.33	7.75	9.95	9.56	10.14	8.15
	10	5.89	6.55	8.50	7.00	9.30	10.09	11.98	9.71
Mean [t.u.]		5.93	8.16	8.99	7.48	9.88	10.38	11.48	8.82
Average time		6	8	9	7.5	10	10.5	11.5	9

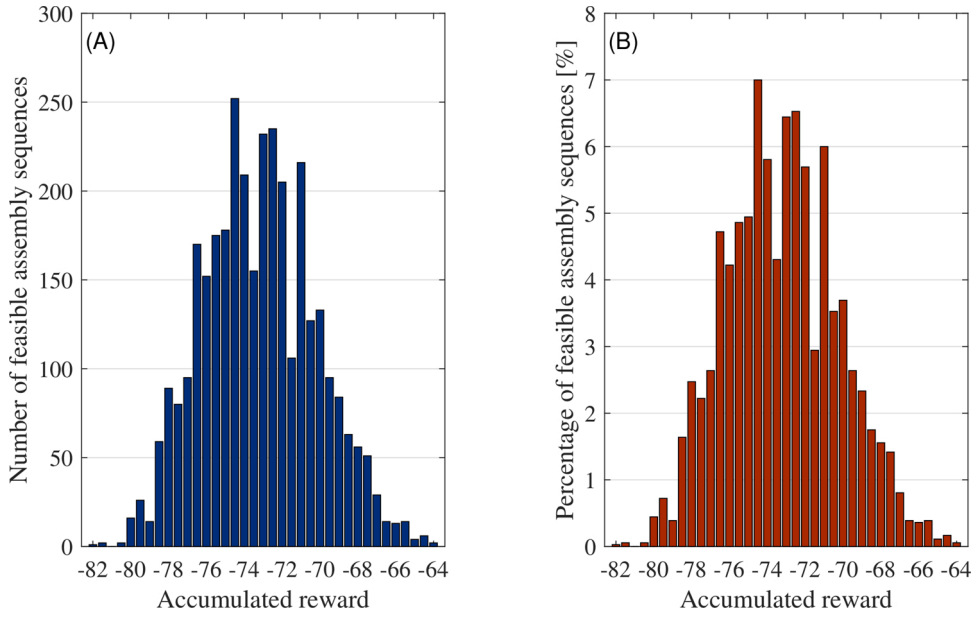


Fig. 13. Distribution by number (A) and percentage (B) of feasible assembly sequences' accumulated rewards.

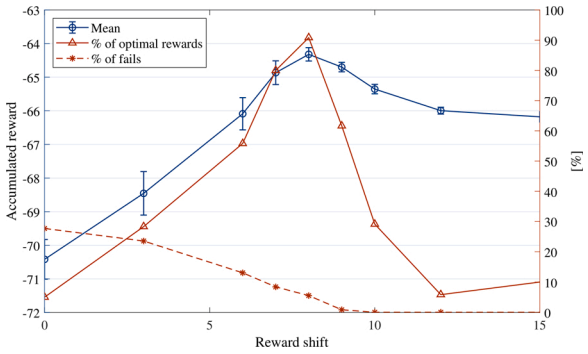


Fig. 14. Reward shift's impact on the agent's performance.

task reward, but it may be relevant to confirm this relationship with a different scenario. A value of the reward shift slightly lower than the mean task reward may improve the mean accumulated reward since a larger number of the accumulated rewards are negative. However, it also increases the percentage of fails, which is highly prejudicial for a real scenario. For that reason, the optimal value for the reward shift is 9. With this new r_s value, the learning rate and the discount factor were individually analysed (Fig. 15).

Since in both cases the confidence interval is very small, even small differences in the mean are significant. Both in the learning rate and in the discount factor, it can be concluded that the optimal value is 1, however, in the discount factor case the difference in the mean and in the percentage of optimal results is more accentuated. Then, with the same set's parameters as before, the maximum steps per episode were individually analysed (Fig. 16).

It is possible to conclude that an early increase in the maximum

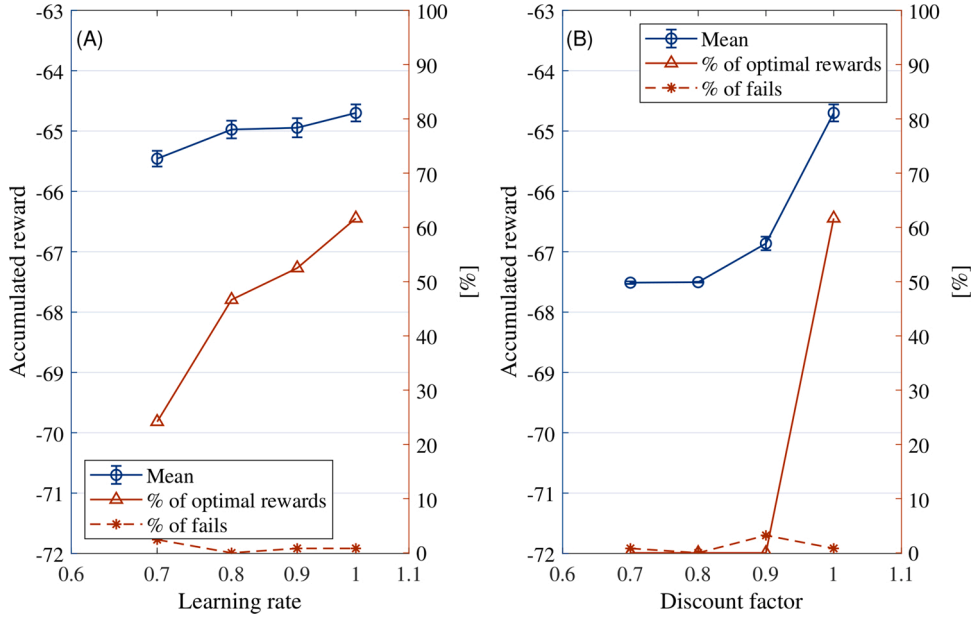


Fig. 15. Learning rate's and discount factor's impact on the agent's performance.

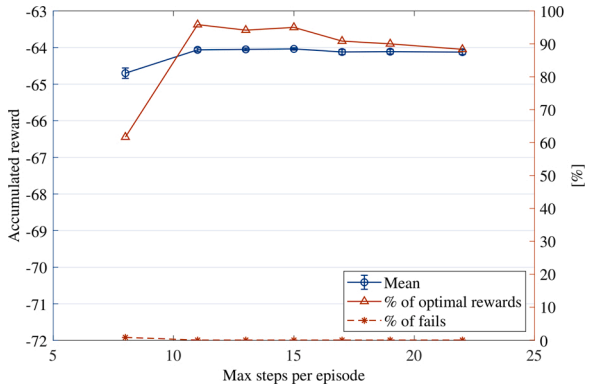


Fig. 16. Maximum steps per episode's impact on the agent's performance.

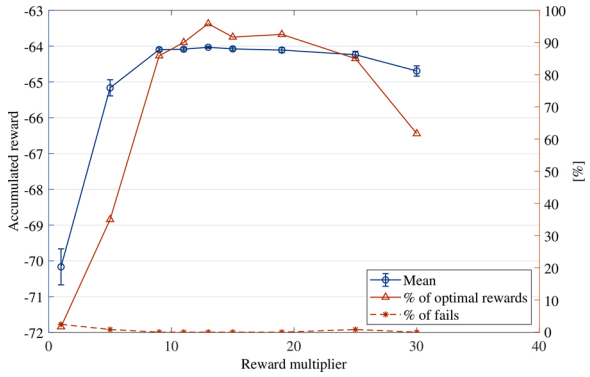


Fig. 17. Reward multiplier's impact on the agent's performance.

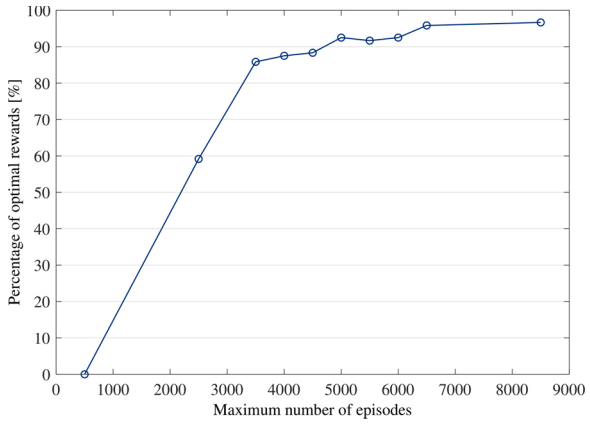


Fig. 18. Evolution of the percentage of optimal rewards in regards to the maximum number of episodes.

number of steps per episode leads to a significant increase both in the mean and in the percentage of optimal rewards. This shows that additional steps beyond the number of assembly tasks slightly contributes to improve the agent learning process, although further increase in this value does not significantly alter the results. The value 15 was selected for this parameter.

Lastly, with all the other parameters decided, the reward multiplier's impact was studied with various values (1, 5, 9, 11, 13, 15, 19, 25, 30) and the experiments results are visible in Fig. 17.

It can be observed that there is an optimal value for r_m, since the increase in the value of the reward multiplier leads to a steady increase both in the mean and in the percentage of optimal rewards. For the smaller values of the reward multiplier, the percentage of fails is

Table 12
Optimal set's parameters.

Parameter	Value
Learning rate	1
Discount factor	1
Epsilon	0.9
Epsilon decay	0.00005
Max steps per episodes	15
Max episodes	6500
Reward shift	9
Reward multiplier	13
Reward penalty	−1,000,000

nonzero. Based on the graph, it can be defined that the optimal reward multiplier value is 13. To confirm the choice of 6500 maximum number of episodes, in Fig. 18 are plotted the agent's performances for various values of maximum number of episodes.

As expected, a reduction in the maximum number of episodes leads to a lower percentage of optimal rewards. Therefore, in order to guarantee the threshold of 95% the value of the maximum number of episodes is maintained. After the previous scenarios experiments, it is possible to conclude that the best set's parameters are the ones expressed in Table 12, with which the agent, in 120 experiments with 6500 episodes, was able to learn one of the 2 optimal assembly sequences 115 times (≈ 95.8 %) (Fig. 18), one of the assembly sequences with the second best accumulated reward 4 times (≈ 3.3 %) and one of the fifth best accumulated reward once (≈ 0.8 %), while never failing to learn a feasible assembly sequence.

3.6. Scenario III: Comparison of the learning of an assembly sequence based on measured task average times and estimated variances with PPO

PPO was implemented in order to compare its performance with the Q-learning algorithm. However, it is important to notice that the PPO implementation and optimisation was not as thorough as in the Q-learning. The Q-learning environment was reused in the PPO implementation, while the actor and critic networks were created in accordance to the environment's action and observation specifications. The actor and critic MLPs (multi layer perceptrons) had the structures displayed in Fig. 19.

In order to achieve a more stable learning, the rewards were normalized based on the set of Eq. (11), the impossible actions were not penalised and the discount factor was decreased to a value slightly lower than 1. It was verified that the penalty of impossible actions destabilized the learning phase and, therefore, a different method should be used to discourage or restrict the agent to select them. Also due to the normalization, the reward shift parameter was not used in the PPO implementation.

$$\begin{cases} R(s,a) = r_c + r_m \times \dfrac{max(T(s,a)) - T(s,a)}{max(T(s,a)) - min(T(s,a))} & \text{if last possible action} \\ R(s,a) = r_m \times \dfrac{max(T(s,a)) - T(s,a)}{max(T(s,a)) - min(T(s,a))} & \text{if possible action} \\ R(s,a) = 0 & \text{if impossible action} \end{cases} \tag{11}$$

The average episodic reward obtained after the optimization of the PPO's hyperparameters of three distinct learning attempts can be visualized in Fig. 20.

Analysing Fig. 20 it is possible to conclude that the agent is able to learn and reach accumulated rewards of approximately −70 (the values were associated to the effective time duration to allow a comparison). A further optimization of the parameters could lead to better results, similar to the Q-learning algorithm, however, while the Q-learning algorithm only required 6500 episodes, the PPO algorithm exceeds 20,000 episodes.

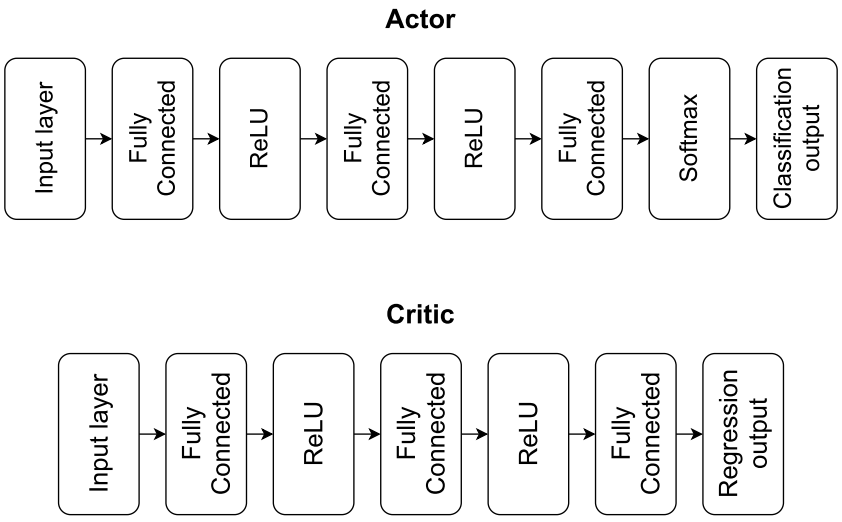


Fig. 19. Actor and critic structures.

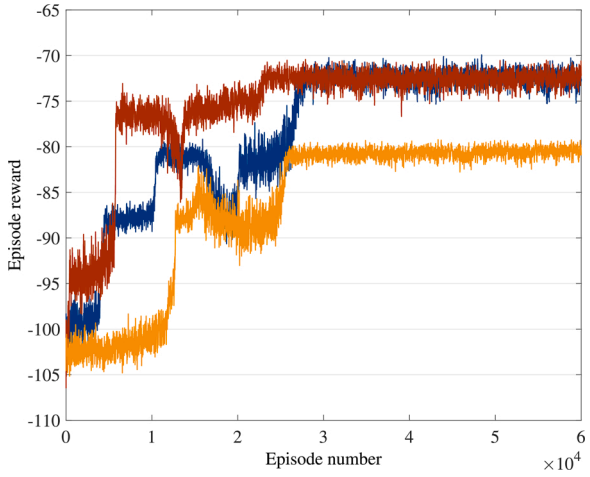


Fig. 20. Evolution of the average episodic reward in three distinct training attempts.

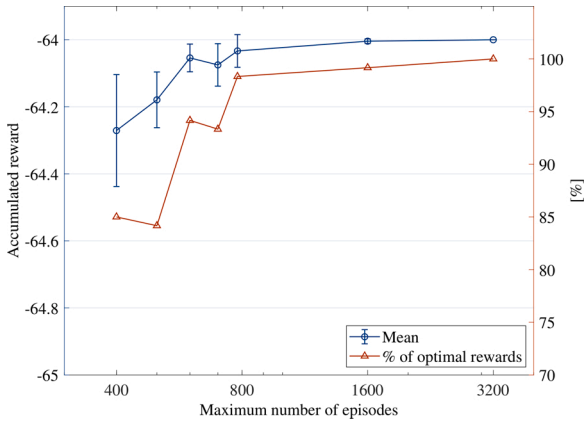


Fig. 21. Evolution of the mean accumulated reward and the percentage of optimal rewards in regards to the maximum number of episodes.

3.7. Scenario IV: Improving the learning of an assembly sequence with the Q-learning algorithm based on MDP restricted actions

As previously discussed, with a general MDP structure with all states of the problem, the agent is able to select impossible actions which are penalised. However, this greatly increases the exploration required to achieve a feasible assembly sequence. Therefore, in this final scenario, the impossible actions were now restricted in the MDP structure (instead of the imposed reward penalty) to identify the possibility to reduce the number of episodes required for learning. Apart from the maximum number of episodes, which were set to (400, 500, 600, 700, 750, 780, 1600, 3200), the epsilon decay, which had the corresponding values calculated from the regression equation in Fig. 11, and the reward penalty for impossible tasks that is now nonexistent, the set of parameters used in this experiments were the same displayed in Table 12.

It can be observed in Fig. 21 that by restricting impossible actions, the maximum number of episodes can be reduced to 780 episodes, which corresponds to only 21.7% of all possible assembly sequences, while improving a percentage of optimal rewards to 98.3%.

4. Conclusion and future work

In this work, the challenges in the application of a reinforcement learning algorithm in a assembly sequence problem are studied, considering the implementation of a Q-Learning model-free algorithm. By formulating the problem as a MDP, it is shown that Q-Learning finds an optimal state-action policy that maximises the accumulated reward over a succession of given steps. This allows to verify the application of a scalable method to address the optimisation of an assembly sequence of an object as a sequential decision process, where the action in one state influences the transition to the subsequent state. Despite its recent application in the literature, RL methods show a straightforward versatility in complex problems where uncertainty plays a significant role, such as on-line industrial environments, where until now mostly traditional exact and non-exact optimisation approaches are considered. The improvement efficiency of the assembly time of complex products is often impractical due to the complexity of assessing all tasks combinations. This approach has the advantage of achieving suitable optimisation results, where the optimal assembly sequence was learned 98.3% of the times. To guarantee this threshold (over 95%), the algorithm requires 780 assemblies (episodes) to correctly learn the best assembly sequence, which is corresponds to approximately 21.7% of the number of feasible assembly sequences. It is acknowledged that an increasingly complex assembly processes might reveal an unpractical use of Q-Learning algorithm due to the "curse of dimensionality", which suggests to explore alternative RL methods.

In future work, the acquisition of the tasks' durations on-line during real assemblies will be considered. The system will have an intuitive

graphical user interface (installed on the factory floor) to facilitate the interaction of the human worker with the system. In addition, we are working on a CAD-based solution to automate the process to define the individual parts and tools that serve as input to the RL algorithm.

Funding

This research was partially supported by project PRODUTECH4S&C (46102) by UE/FEDER through the program COMPETE 2020 and the Portuguese Foundation for Science and Technology (FCT): COBOTIS (PTDC/EME-EME/32595/2017) and UIDB/00285/2020.

Declaration of Competing Interest

The authors report no declarations of interest.

References

[1] Vieira M, Pinto-Varela T, Barbosa-Póvoa AP. A model-based decision support framework for the optimisation of production planning in the biopharmaceutical industry. Comput Ind Eng 2019;129:354–67.
[2] Sutton RS, Barto AG. Reinforcement learning: an introduction. second edition. The MIT Press; 2018.
[3] Silver D, Huang A, Maddison CJ, Guez A, Sifre L, van den Driessche G, et al. Mastering the game of go with deep neural networks and tree search. Nature 2016; 529(January (7587)):484–9.
[4] Silver D, Schrittwieser J, Simonyan K, Antonoglou I, Huang A, Guez A, et al. Mastering the game of go without human knowledge. Nature 2017;550(7676 (Oct)):354–9.
[5] Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, et al. Human-level control through deep reinforcement learning. Nature 2015;518(Feb (7540)): 529–33.
[6] Fernandes S. Reinforcement learning para problemas de otimização. University of Porto; 2019.
[7] Wang Y-C, Usher JM. A reinforcement learning approach for developing routing policies in multi-agent production scheduling. Int J Adv Manuf Technol 2007;33 (June (3)):323–33.
[8] Shahrabi J, Adibi MA, Mahootchi M. A reinforcement learning approach to parameter estimation in dynamic job shop scheduling. Comput Ind Eng 2017;110: 75–82.
[9] Leng J, Jin C, Vogl A, Liu H. Deep reinforcement learning for a color-batching resequencing problem. J Manuf Syst 2020;56:175–87.
[10] Huang J, Chang Q, Chakraborty N. Machine preventive replacement policy for serial production lines based on reinforcement learning. 2019 IEEE 15th international conference on automation science and engineering (CASE) 2019: 523–8.
[11] Paraschos PD, Koulinas GK, Koulouriotis DE. Reinforcement learning for combined production-maintenance and quality control of a manufacturing system with deterioration failures. J Manuf Syst 2020;56:470–83.
[12] Ou X, Chang Q, Arinez J, Zou J. Gantry work cell scheduling through reinforcement learning with knowledge-guided reward setting. IEEE Access 2018; 6:14699–709.
[13] Ou X, Chang Q, Chakraborty N. Simulation study on reward function of reinforcement learning in gantry work cell scheduling. J Manuf Syst 2019;50:1–8.
[14] Watanabe K, Inada S. Search algorithm of the assembly sequence of products by using past learning results. Int J Prod Econ 2020;226:107615.
[15] Kim YG, Lee S, Son J, Bae H, Do Chung B. Multi-agent system and reinforcement learning approach for distributed intelligence in a flexible smart manufacturing system. J Manuf Syst 2020;57:440–50.
[16] Leng J, Ruan G, Song Y, Liu Q, Fu Y, Ding K, et al. A loosely-coupled deep reinforcement learning approach for order acceptance decision of mass-individualized printed circuit board manufacturing in industry 4.0. J Clean Prod 2021;280:124405.
[17] Tsai Y-T, Lee C-H, Liu T-Y, Chang T-J, Wang C-S, Pawar SJ, et al. Utilization of a reinforcement learning algorithm for the accurate alignment of a robotic arm in a complete soft fabric shoe tongues automation process. J Manuf Syst 2020;56: 501–13.
[18] Akkaladevi SC, Plasch M, Maddukuri S, Eitzinger C, Pichler A, Rinner B. Toward an interactive reinforcement based learning framework for human robot collaborative assembly processes. Front Robot AI 2018;5:126.
[19] Oliff H, Liu Y, Kumar M, Williams M, Ryan M. Reinforcement learning for facilitating human-robot-interaction in manufacturing. J Manuf Syst 2020;56: 326–40.
[20] Xia K, Sacco C, Kirkpatrick M, Saidy C, Nguyen L, Kircaliali A, et al. A digital twin to train deep reinforcement learning agent for smart manufacturing plants: Environment, interfaces and intelligence. J Manuf Syst 2020.
[21] Hu L, Liu Z, Hu W, Wang Y, Tan J, Wu F. Petri-net-based dynamic scheduling of flexible manufacturing system via deep reinforcement learning with graph convolutional network. J Manuf Syst 2020;55:1–14.
[22] Leng J, Yan D, Liu Q, Zhang H, Zhao G, Wei L, et al. Digital twin-driven joint optimisation of packing and storage assignment in large-scale automated high-rise warehouse product-service system. Int J Comput Integr Manuf 2019:1–18.
[23] Liu Q, Leng J, Yan D, Zhang D, Wei L, Yu A, et al. Digital twin-based designing of the configuration, motion, control, and optimization model of a flow-type smart manufacturing system. J Manuf Syst 2020.
[24] Leng J, Zhang H, Yan D, Liu Q, Chen X, Zhang D. Digital twin-driven manufacturing cyber-physical system for parallel controlling of smart workshop. J Ambient Intell Human Comput 2019;10(March (3)):1155–66.
[25] Kober J, Bagnell JA, Peters J. Reinforcement learning in robotics: a survey. Int J Robot Res 2013;32(11):1238–74.
[26] Bellman RE. Dynamic programming. USA: Dover Publications, Inc.; 2003.
[27] Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. Proximal policy optimization algorithms. 2017 (arXiv preprint), arXiv:1707.06347.
[28] Calli B, Singh A, Walsman A, Srinivasa S, Abbeel P, Dollar AM. The ycb object and model set: Towards common benchmarks for manipulation research. 2015 international conference on advanced robotics (ICAR) 2015:510–7.
[29] Calli B, Walsman A, Singh A, Srinivasa S, Abbeel P, Dollar AM. Benchmarking in manipulation research: using the yale-cmu-berkeley object and model set. IEEE Robot Autom Mag 2015;22(3):36–52.